

Block Round

La Matemática del Proyecto

Documentación técnica y didáctica de los fundamentos matemáticos empleados en el generador de círculos, elipses, esferas y elipsoides *renderizados con texturas de bloques de Minecraft*. Para cada concepto se presentan la definición formal, la justificación geométrica, la correspondencia directa con el código fuente del proyecto y la traducción práctica a los comandos de construcción en el juego (`/fill`, `/clone`, `//sphere`).

Áreas abordadas: geometría analítica · rasterización discreta · topología digital · cálculo integral · álgebra lineal · trigonometría · aritmética de inventario · mapeo de texturas · simetría octaédrica.

Índice

1	Visión general: naturaleza matemática del problema	3
2	Sistema de coordenadas y la malla discreta de voxels	4
3	La ecuación fundamental: círculo y elipse	5
4	Algoritmo Euclidiano (test de distancia)	5
5	Algoritmo de Bresenham (punto medio entero)	7
5.1	Caso circular	7
5.2	Caso elíptico (dos regiones)	8
6	Algoritmo de Umbral (cobertura de esquina)	8
7	Modos de renderizado: Relleno, Fino, Grueso	9
7.1	Relleno	9
7.2	Fino (contorno)	9
7.3	Grueso	9
8	Cálculo de área discreta	10
9	De 2D a 3D: la ecuación del elipsoide	10
10	Voxelización y cálculo de volumen	11
11	Cortes geométricos: planos y el corte diagonal a 45°	12
12	Modo Grueso 3D: tridimensionalización por rebanadas	12
13	Cámara 3D: coordenadas esféricas	13
14	Auto-zoom: esfera envolvente y FOV	13
15	Sombreado y multiplicación de textura	14
16	Transición: de la matemática continua a la construcción in-game	14
17	Aritmética del inventario Minecraft	16
17.1	Slot único, stack único, pack lleno	16
17.2	Almacenamiento: cofres, cofres dobles, shulker boxes	16
17.3	Tabla de referencia: esferas canónicas	16

18 Highlight overlay y conteo de caras expuestas	17
18.1 Qué cuenta como cara expuesta	17
18.2 Fórmula de detección de borde	17
18.3 Conteo de caras expuestas como heurística de supervivencia . . .	17
19 Construcción capa por capa (descomposición horizontal)	18
20 Optimización por simetría octaédrica (O_h)	19
20.1 Secuencia concreta de comandos	19
20.2 Comparación de tiempo	20
21 Texturas de bloques y composición visual	20
21.1 Las seis caras y el mapeo UV	20
21.2 Familias de bloques y tonos	21
21.3 Gradientes por mezcla de bloques	21
22 Conclusión	21
Apéndice A: Tablas de referencia de construcción y ejemplo práctico	23

1. Visión general: naturaleza matemática del problema

Block Round es un generador de formas redondeadas *texturizadas con bloques de Minecraft*: círculos y elipses en 2D, esferas y elipsoides en 3D. El problema central pertenece a la *rasterización discreta*: dada una curva o superficie definida analíticamente sobre los reales $\mathbb{R}$, determinar qué celdas de una malla regular (píxeles en 2D, voxels en 3D) deben marcarse para representarla. Cada celda marcada en Block Round recibe además una textura de seis caras del catálogo de Minecraft (Bloque de Hierba, Adoquín, Tablones de Roble, Cuarzo, Diamante, etc.), de modo que la figura resultante no es solo una silueta matemática sino una *estructura construible* que el usuario puede replicar en supervivencia, en creativo con `/fill`, o importar mediante Sponge Schematic v2 (`.schem`) en WorldEdit, Litematica o MCEdit.

El problema no admite solución única. Distintos criterios de inclusión producen distintas siluetas discretas, y cada criterio tiene su propia justificación matemática. Por eso el proyecto implementa tres algoritmos 2D distintos (Euclidiano, Bresenham, Umbral), tres modos de renderizado (Relleno, Fino, Grueso) y tres estilos de sombreado 3D. La elección de algoritmo se decide no solo por la suavidad visual sino también por el número de bloques que el usuario necesitará reunir — una esfera de diámetro $D = 20$ requiere 4189 bloques con Euclidiano pero hasta 4322 con Umbral.

La ejecución ocurre íntegramente en el navegador, sin servidor, lo que impone un requisito adicional de *eficiencia computacional* (una esfera de Diamante de $D = 32$ debe renderizarse a 60 fps incluyendo 17156 voxels texturizados). Las áreas teóricas movilizadas incluyen:

- geometría analítica de cónicas y cuádricas;
- algoritmos incrementales con aritmética entera (Bresenham);
- topología digital (vecindarios 4-, 6- y 26-conectados);
- cálculo integral y medida de conteo;
- álgebra lineal (planos de corte, proyección perspectiva);
- trigonometría y coordenadas esféricas;
- aritmética combinatoria de inventario y simetría de grupos.

Novedades respecto al proyecto hermano Pixel Round. Block Round

añade cinco preocupaciones matemáticas nuevas que solo surgen cuando las celdas renderizadas son *bloques de Minecraft*: (i) el mapeo UV de las seis caras de cada voxel; (ii) la aritmética del inventario de supervivencia (packs de 64 ítems, cofres simples de 27 slots, cofres dobles de 54 slots); (iii) el overlay de contorno usado para contar caras expuestas durante la construcción; (iv) la descomposición de una forma 3D en capas horizontales, dado que la construcción real procede planta por planta; (v) la explotación del grupo de simetría octaédrico O_h para reducir el coste de colocación manual por un factor de 8 mediante `/clone`.

2. Sistema de coordenadas y la malla discreta de voxels

El canvas es una malla de $G_x \times G_y$ celdas en 2D, extendida a $G_x \times G_y \times G_z$ voxels en 3D. Cada celda (i, j) ocupa el cuadrado unitario $[i, i + 1] \times [j, j + 1]$; en 3D cada voxel ocupa el cubo unitario. En código continuo, el **centro** de la celda está en $(i + \frac{1}{2}, j + \frac{1}{2})$. Esta convención es crítica: comparar la circunferencia con la *esquina* (i, j) o con el *centro* cambia qué celdas pertenecen a la forma y, por tanto, qué bloques debe colocar el usuario.

$$\text{centro de la celda } (i, j) = (i + \frac{1}{2}, j + \frac{1}{2})$$

Para una forma de diámetro D , el código expande la malla a $D + 2$ celdas en cada eje (margen de una celda a cada lado). Esto garantiza que los píxeles extremos (N, S, E, O) nunca caigan en el límite de la matriz. El **centro** de la forma está en $(G_x/2, G_y/2)$, y los *semi-ejes* son $r_x = D/2$ y $r_y = D/2$ (o $W/2$ y $H/2$ para elipse). En 3D la misma lógica se extiende con margen en el eje z . La celda unitaria del canvas coincide con el bloque unitario de Minecraft, por lo que las coordenadas del algoritmo se mapean uno a uno a las coordenadas (x, y, z) del juego.

Mapeo a Minecraft. Si el jugador aparece en $(0, 64, 0)$ y desea una esfera $D = 16$ ($r = 8$) apoyada en el suelo, el centro debe situarse en $(0, 72, 0)$ (8 bloques sobre la superficie), de modo que el polo sur toque el piso. Comando WorldEdit: `//sphere stone 8` estando en $(0, 72, 0)$.

3. La ecuación fundamental: círculo y elipse

Toda la teoría del proyecto parte de la ecuación implícita de la elipse centrada en el origen con semi-ejes r_x y r_y :

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 = 1$$

Cuando $r_x = r_y = r$, la elipse degenera en una **circunferencia** de radio r . Los puntos del *interior* satisfacen ≤ 1 ; los del *exterior*, > 1 . Esa desigualdad define si un voxel pertenece a la forma y, por tanto, si el usuario coloca un bloque allí.

Trasladando el centro a (c_x, c_y) y evaluando en el centro de cada celda:

$$d_x = \frac{i + \frac{1}{2} - c_x}{r_x}, \quad d_y = \frac{j + \frac{1}{2} - c_y}{r_y}, \quad \text{dentro} \iff d_x^2 + d_y^2 \leq 1$$

Mapeo a Minecraft. Para una torre en supervivencia, el usuario suele querer un *círculo aplanado ancho* en la base y círculos cada vez menores a medida que la torre sube. Cada planta es una aplicación de la ecuación implícita con los mismos semi-ejes y distinto z . Block Round computa la capa 2D una vez y el usuario duplica con `/clone` por planta.

4. Algoritmo Euclidiano (test de distancia)

Idea. Para cada fila j , encontrar analíticamente las columnas i dentro de la elipse. Resolviendo la ecuación para x , dado y :

$$x = c_x \pm r_x \sqrt{1 - \left(\frac{y - c_y}{r_y}\right)^2}$$

Para una fila j cuyo desplazamiento vertical es $d_y = j + \frac{1}{2} - c_y$, la *semi-anchura horizontal* de la elipse a esa altura es:

$$dxM = r_x \sqrt{1 - \frac{d_y^2}{r_y^2}}$$

Si $d_y^2/r_y^2 \geq 1$, la fila no corta la elipse y se omite. En caso contrario, se llenan todas las columnas i tales que $c_x - dxM \leq i + \frac{1}{2} \leq c_x + dxM$. Los límites enteros salen de:

$$lo = \lceil c_x - dxM - \frac{1}{2} \rceil, \quad hi = \lfloor c_x + dxM - \frac{1}{2} \rfloor$$

Justificación. El término $+\frac{1}{2}$ resulta de la convención de que la celda i tiene centro en $i + \frac{1}{2}$. Las funciones $\lceil \cdot \rceil$ y $\lfloor \cdot \rfloor$ convierten el intervalo continuo a índices enteros, garantizando que solo las celdas cuyo **centro** pertenece al interior se incluyan. Esta formulación produce la aproximación discreta más próxima a la curva continua y, por tanto, la silueta más suave visualmente. Para diámetros pequeños, sin embargo, el resultado puede tener un carácter pixel-art menos pronunciado — un círculo Euclidiano de $D = 5$ tiene solo 13 bloques, mientras que el de Umbral tiene 21.

Mapeo a Minecraft. El criterio Euclidiano es el más cercano a lo que produce `//sphere` en WorldEdit. Para una esfera de Adoquín $D = 16$ ($r = 8$) usando solo la cáscara Euclidiana, el volumen es ≈ 2145 bloques en modo relleno, ≈ 804 en modo fino — la diferencia entre reunir 34 packs (2176 bloques) y 13 packs (832 bloques).

5. Algoritmo de Bresenham (punto medio entero)

El algoritmo de Bresenham, publicado en 1965 para trazar segmentos, fue extendido a circunferencias y generalizado por Pitteway y Kappel a elipses arbitrarias. Su propiedad distintiva es prescindir de operaciones en coma flotante y de raíces cuadradas: la determinación del siguiente píxel usa exclusivamente **sumas y comparaciones enteras**, mediante una *función de decisión* que evalúa de qué lado de la curva analítica está el punto medio entre los dos píxeles candidatos. Para construcción en Minecraft es el algoritmo que entrega el look pixel-art más *reconocible* — la misma silueta escalonada que la comunidad construye a mano desde 2011.

5.1 Caso circular

Para una circunferencia de radio entero R , se parte de $(x=0, y=R)$ con parámetro de decisión inicial:

$$d_0 = 1 - R$$

En cada paso (en el octante de 0 a 45°):

$$\begin{cases} \text{si } d < 0 : & \text{el siguiente es } (x+1, y), \quad d += 2x + 3 \\ \text{si } d \geq 0 : & \text{el siguiente es } (x+1, y-1), \quad d += 2(x - y) + 5 \end{cases}$$

Justificación. La función d aproxima, en cada iteración, el valor de la ecuación implícita de la circunferencia evaluado en el **punto medio** entre las dos opciones de píxel candidato:

$$F\left(x + 1, y - \frac{1}{2}\right) = (x + 1)^2 + \left(y - \frac{1}{2}\right)^2 - R^2$$

El signo de d indica si el punto medio está dentro de la circunferencia (avanzar solo en horizontal) o fuera (decrementar también y). Los ocho octantes se obtienen aplicando las simetrías del grupo diedral D_4 , lo que corresponde, en código, a las ocho llamadas a `span(...)`. El trazado resultante exhibe el patrón escalonado característico de las aproximaciones discretas de curvas suaves.

5.2 Caso elíptico (dos regiones)

Para una elipse con semi-ejes a, b , el algoritmo divide el primer cuadrante en **dos regiones**, delimitadas por el punto donde la tangente forma 45° :

Región I: $2b^2x < 2a^2y$ ($|dy/dx| < 1$), Región II: en caso contrario

Los parámetros de decisión iniciales son:

$$p_1 = b^2 - a^2b + \frac{a^2}{4},$$

$$p_2 = b^2\left(x + \frac{1}{2}\right)^2 + a^2(y - 1)^2 - a^2b^2.$$

En cada iteración, p se actualiza con incrementos pre-calculados (todos enteros tras multiplicar por 4). El resultado es la mínima aritmética posible por píxel: *un test de signo y dos sumas*.

Mapeo a Minecraft. Para constructores de supervivencia la silueta Bresenham suele ser preferible a la Euclidiana porque los pasos discretos son fáciles de contar a mano — el usuario sabe que el siguiente bloque es siempre *recto* o *diagonal*, lo que mantiene el gesto consistente. Un círculo Bresenham de $D = 11$ tiene la secuencia canónica de 80 bloques publicada en decenas de guías.

6. Algoritmo de Umbral (cobertura de esquina)

Este algoritmo pregunta: *¿hay alguna esquina de la celda dentro de la elipse?* Para cada celda (i, j) el código localiza la esquina más cercana al centro de la forma (proyección del centro sobre las aristas de la celda):

$$n_x = \text{clamp}(c_x, i, i+1), \quad n_y = \text{clamp}(c_y, j, j+1)$$

$$\text{dentro} \iff \left(\frac{n_x - c_x}{r_x}\right)^2 + \left(\frac{n_y - c_y}{r_y}\right)^2 \leq 0,94$$

El valor **0,94** es un *umbral empírico* ligeramente inferior a 1,0. Su función es eliminar el artefacto conocido como *spike tangencial* de 1 píxel que aparece en los cuatro puntos cardinales (N, S, E, O), donde la curva es tangente a una arista entera de la celda. Sin esta reducción el algoritmo incluiría una celda extra cuya contribución no mejora la representación visual.

De los tres algoritmos este produce la silueta de mayor área para el mismo diámetro D , ya que la condición de inclusión es la menos restrictiva. Para Minecraft, da el

aspecto más *redondeado* a costa de más bloques — $D = 16$ produce 213 bloques de cáscara con Umbral frente a 200 con Bresenham y 192 con Euclidiano.

Mapeo a Minecraft. Umbral es la elección cuando la construcción se verá desde lejos (p.ej. una cúpula de Cuarzo sobre un templo), porque la silueta ligeramente mayor se lee mejor a través de las sombras de oclusión ambiental. Trade-off: aproximadamente 5–10% más bloques por cáscara que Euclidiano.

7. Modos de renderizado: Relleno, Fino, Grueso

Con una matriz $f[j][i] = 1$ para celdas interiores, el proyecto deriva tres estilos visuales por **topología discreta**:

7.1 Relleno

Devuelve f sin cambios. Todas las celdas internas son bloques. Es lo que produce `//sphere stone 8`.

7.2 Fino (contorno)

Una celda pertenece al contorno fino si está rellena *y* tiene al menos un vecino ortogonal (N, S, E, O) *fuera* de la forma. En notación lógica:

$$b(i, j) = f(i, j) \wedge (\neg f(i-1, j) \vee \neg f(i+1, j) \vee \neg f(i, j-1) \vee \neg f(i, j+1))$$

Geométricamente: es el **borde discreto** de la forma, análogo discreto de la frontera topológica $\partial F = F \cap \overline{F^c}$. En Minecraft equivale a `//hsphere`: solo se materializa la cáscara.

7.3 Grueso

El modo grueso añade a los píxeles de borde los **puentes diagonales**: celdas internas que tienen simultáneamente un vecino-borde horizontal y uno vertical. Esto cierra las diagonales de 1 píxel que el modo fino deja abiertas, útil cuando el contorno debe parecer estable en escena (sin agujeros a 45°) — particularmente importante para cúpulas de Vidrio y Hielo.

$$\begin{aligned} t(i, j) &= b(i, j) \vee (f(i, j) \wedge h_H \wedge h_V), \\ h_H &= b(i-1, j) \vee b(i+1, j), \\ h_V &= b(i, j-1) \vee b(i, j+1). \end{aligned}$$

Mapeo a Minecraft. Para una cúpula de Vidrio de $D = 20$ (hueca), el modo grueso da una cáscara *estanca* de 608 bloques frente a ≈ 510 del modo fino — los ~ 100 extra son costura diagonal que impide que la luz ambiental fugue por las grietas diagonales.

8. Cálculo de área discreta

La función `area2D` simplemente **cuenta** las celdas marcadas en f . Es la *medida de conteo*, que aproxima la integral de Riemann de la función indicadora de la elipse:

$$\begin{array}{ccc} \pi r_x r_y & \longleftrightarrow & \sum_{j=0}^{G_y-1} \sum_{i=0}^{G_x-1} f(i, j) \\ \text{área continua} & & \text{área discreta} \end{array}$$

Para D grande, área discreta/área continua $\rightarrow 1$. Para D pequeño, la diferencia entre algoritmos es visible: Umbral tiende a sobrestimar; Euclidiano queda cerca del área ideal; Bresenham, intermedio.

Mapeo a Minecraft. El chip de conteo de bloques de Block Round muestra exactamente ese número: 268 para $D = 8$, 904 para $D = 12$, 2145 para $D = 16$. El chip expresa el conteo como N (*stacks* $\times 64$ + *resto*) — por ejemplo $2145 = 33 \times 64 + 33$, es decir, «33 stacks más 33 ítems», que es lo que el usuario debe reunir.

9. De 2D a 3D: la ecuación del elipsoide

En 3D, la forma fundamental es el **elipsoide** centrado en (c_x, c_y, c_z) con semi-ejes r_x, r_y, r_z . Su ecuación generaliza la elipse:

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 + \left(\frac{z}{r_z}\right)^2 = 1$$

Cuando $r_x = r_y = r_z$ recuperamos la **esfera**. Evaluando en el centro de cada voxel:

$$a_\xi = \frac{\xi + \frac{1}{2} - c_\xi}{r_\xi} \quad (\xi \in \{x, y, z\}), \quad \text{dentro} \iff a_x^2 + a_y^2 + a_z^2 \leq 1$$

Mapeo a Minecraft. Un elipsoide $W = 20, H = 10, D = 20$ es una *esfera aplanada* ideal para una cúpula subterránea o un techo de estadio. Volumen ≈ 2094 bloques (33 packs). Comando WorldEdit: `//ellipsoid stone 10 5 10`.

10. Voxelización y cálculo de volumen

El volumen continuo del elipsoide es un resultado clásico del cálculo integral, obtenido por integración sobre discos:

$$V = \frac{4}{3} \pi r_x r_y r_z$$

La versión discreta recorre cada voxel de la caja $D_x \times D_y \times D_z$ e incrementa un contador cuando $a_x^2 + a_y^2 + a_z^2 \leq 1$. Es el algoritmo `voxelVolume`.

Cáscara vs. interior. En modo *filled* el proyecto muestra solo voxels con al menos un vecino-6 fuera del conjunto preservado (voxels visibles desde la superficie externa o la cara de corte). En *thin* solo la **superficie** del elipsoide completo (1 voxel de espesor). En supervivencia, solo las caras visibles necesitan el bloque texturizado dedicado — los voxels interiores pueden ser relleno barato (Tierra en lugar de Diamante) para ahorrar recursos.

Tabla de referencia. La tabla resume los diámetros canónicos de esfera más solicitados, traducidos en número de bloques (V), packs de 64 ($P = \lceil V/64 \rceil$) y cofres dobles de 3456 slots ($DC = \lceil V/3,456 \rceil$). Sirve para planear la recolección antes de construir.

Diámetro	Bloques (V)	Packs ($\times 64$)	Cofres dobles	Nota
D=8	268	5	1	cúpula pequeña
D=12	904	15	1	esfera media
D=16	2145	34	1	cima de torre
D=20	4189	66	2	techo de estadio
D=24	7238	114	3	cúpula grande
D=32	17156	269	5	mega-build

Volúmenes en modo relleno calculados con Euclidiano en $D \in \{8, 12, 16, 20, 24, 32\}$. Los valores en modo fino (cáscara) son aproximadamente $V_{\text{cáscara}} \approx 4\pi r^2$ bloques de un voxel de espesor, es decir, entre 25% y 40% del volumen relleno según D .

11. Cortes geométricos: planos y el corte diagonal a 45°

Block Round permite cortar el sólido por **tres planos**. Cada corte es una *desigualdad lineal* que descarta voxels:

$$\begin{aligned} \text{Eje X:} \quad & x \geq \textit{cut} \quad \Rightarrow \text{descartado} \\ \text{Eje Y:} \quad & y \geq \textit{cut} \quad \Rightarrow \text{descartado} \\ \text{Diagonal:} \quad & x + y \geq \textit{cut} \quad \Rightarrow \text{descartado} \end{aligned}$$

Los planos X y Y son perpendiculares a los ejes coordenados, con normales $(1, 0, 0)$ y $(0, 1, 0)$. El *plano diagonal* tiene normal $(1, 1, 0)/\sqrt{2}$ y corta a 45° en el plano XY: es la familia $x + y = k$. Como el máximo de $x + y$ en una caja $D_x \times D_y$ es $D_x + D_y$, el slider diagonal tiene amplitud $D_x + D_y$, mientras X e Y tienen D_x y D_y . El corte es **proporcional**: el código guarda *cutPct* y recalcula el límite como

$$\textit{cut} = \textit{cutPct} \cdot \max_{\text{eje}}$$

de modo que 50% en un eje sigue siendo 50% al cambiar de eje o redimensionar.

Mapeo a Minecraft. El corte Y al 50% sobre una esfera produce la *cúpula* clásica de la mitad del volumen. Para $D = 16$: $V_{\text{full}} = 2145$, $V_{\text{cúpula}} \approx 1095$ bloques (≈ 18 packs). El corte diagonal al 50% produce una sección de *media-fuente* muy usada para interiores de fuentes y graderíos.

12. Modo Grueso 3D: tridimensionalización por rebanadas

Para el modo *thick* en 3D, el proyecto aplica el algoritmo grueso 2D en **todas las rebanadas** en los tres ejes: XY por cada z , XZ por cada y , YZ por cada x . Luego une los resultados. La motivación es la estanqueidad: el conjunto mínimo de voxels que tapan todas las diagonales sin duplicar el espesor de la cáscara. Con $S_z(z)$ la rebanada XY a la altura z y T el operador grueso 2D:

$$\textit{thick3D} = \bigcup_z T(S_z(z)) \cup \bigcup_y T(S_y(y)) \cup \bigcup_x T(S_x(x))$$

El resultado se intersecta con el conjunto preservado por el corte. La teoría es **topología digital**: el operador grueso garantiza *26-conectividad* de la cáscara, útil en Minecraft donde voxels diagonales no se tocan por cara — luz, agua y criaturas atraviesan las grietas diagonales.

Mapeo a Minecraft. Para una cúpula de Vidrio submarina sobre un monumento oceánico, el modo grueso es obligatorio: los puentes diagonales impiden que el agua entre por las grietas que el modo fino deja abiertas. Coste: cúpula $D = 20$ en grueso requiere ≈ 380 bloques de Vidrio frente a ≈ 320 del modo fino — 19% más material a cambio de sellado total.

13. Cámara 3D: coordenadas esféricas

La cámara 3D orbita el objeto a distancia r con dos ángulos — θ (acimut, plano horizontal) y φ (polar, desde el eje vertical). Es la **convención física** de coordenadas esféricas, y la conversión a cartesianas (lo que espera *three.js*) es:

$$\begin{aligned}x &= r \sin(\varphi) \cos(\theta), \\y &= r \cos(\varphi), \\z &= r \sin(\varphi) \sin(\theta).\end{aligned}$$

Posición inicial: $\theta = \pi/4$ (45° , perspectiva isométrica) y $\varphi = \pi/3$ (60° del polo Norte, ligeramente sobre el ecuador). El arrastre del ratón incrementa θ y φ directamente; rueda/pinza incrementa r . La simplicidad de esta parametrización permite la rotación libre sin cuaterniones ni matrices explícitas.

14. Auto-zoom: esfera envolvente y FOV

Cuando el usuario cambia el tamaño de la figura o resetea la cámara, el proyecto recalcula la **distancia ideal** para que el objeto quepa en pantalla en cualquier ángulo. La idea es encerrar el objeto en una *esfera de radio R* (la mitad de la diagonal del AABB, axis-aligned bounding box):

$$R = \sqrt{(D_x/2)^2 + (D_y/2)^2 + (D_z/2)^2}$$

Una esfera tiene **proyección invariante por rotación**, así que si cabe en una orientación, cabe en todas. Dado el FOV vertical ($fov_V = 35^\circ$ en radianes), la distancia para enmarcar una esfera de radio R es:

$$dist_V = \frac{R}{\tan(fov_V/2)}$$

El FOV horizontal sale de la relación de aspecto del canvas ($aspect = \text{ancho/alto}$) por:

$$fov_H = 2 \arctan(\tan(fov_V/2) \cdot aspect)$$

La distancia final es el máximo entre $dist_V$ y $dist_H$, multiplicado por 1,20 (20% de margen visual). Cuando la figura está cortada, el código encoge D_x o D_y al tamaño restante. Cuando el easter egg del roble o el creeper están activos, el bounding box se extiende hacia arriba para incluir la altura de la entidad.

15. Sombreado y multiplicación de textura

Los tres estilos 3D (*Classic*, *Smooth*, *Blocks*) se diferencian por **factores multiplicativos** aplicados a las tres caras visibles de cada voxel (techo, lado iluminado, lado oscuro). La función `shadeHex(hex, factor)` parsea el hex en (R, G, B) y multiplica cada canal con *clamp* en $[0, 255]$ — el resultado es un tinte aplicado sobre la textura cruda del bloque:

$$R' = \text{clamp}(R \cdot f), \quad G' = \text{clamp}(G \cdot f), \quad B' = \text{clamp}(B \cdot f)$$

Es una *aproximación lineal* de la función de iluminación Lambertiana ideal,

$$I = \max(0, \mathbf{n} \cdot \mathbf{l}) \cdot \text{textura},$$

donde los tres factores corresponden al coseno del ángulo entre la normal de cada cara y una dirección de luz fija. En *Smooth* los factores son próximos (caras parecidas); en *Classic* difieren más (contraste fuerte, el look del Minecraft vanilla). *Blocks* introduce un hueco geométrico en los voxels — una traslación constante de cada cara hacia dentro, para revelar las fronteras entre voxels incluso cuando los vecinos comparten textura. Para bloques emisivos el término Lambertiano se sustituye por una constante $I = \text{factor_emisivo} \cdot \text{textura}$.

16. Transición: de la matemática continua a la construcción in-game

Las quince secciones anteriores describen la canalización *continuo-a-discreto* que Block Round comparte con su proyecto hermano Pixel Round: de la ecuación implícita de la elipse a la cámara que encuadra el resultado. Las seis secciones restantes describen lo **específico** de Block Round — la capa adicional de matemática introducida por el hecho de que las celdas renderizadas no son píxeles abstractos sino *bloques de Minecraft*, cada uno con seis caras texturizadas, un peso en el inventario y un coste de tiempo para reunir y colocar.

No son consideraciones cosméticas. La decisión de renderizar con texturas de bloques cambia el problema operativo del usuario: en vez de *exportar un PNG*, el usuario debe *traducir la silueta en un plan de colocación*. El contenido matemático

de las próximas secciones se ocupa exactamente de esa traducción: aritmética de inventario, optimización por conteo de caras, descomposición capa por capa, y explotación de simetría para reducir el coste de N colocaciones a $N/8 + 7$ invocaciones de comando.

17. Aritmética del inventario Minecraft

Un jugador de Minecraft lleva ítems en un *inventario* de 36 slots, organizado como una rejilla 9×4 . Cada slot sostiene hasta 64 ítems del mismo tipo (un *stack* o *pack*). El jugador accede también al *almacenamiento*: cofre simple 27 slots, cofre doble 54 slots. La aritmética que traduce un número-objetivo de bloques en un plan de recolección es por tanto un problema de *división con techo*.

17.1 Slot único, stack único, pack lleno

El *stack* es la unidad elemental de contabilidad de inventario. Cada stack contiene a lo sumo 64 ítems. La conversión de cuenta de bloques N a packs es

17.2 Almacenamiento: cofres, cofres dobles, shulker boxes

Para calcular cuántos *cofres dobles* requiere una construcción, divide la cuenta entre $54 \times 64 = 3\,456$ ítems por cofre doble:

packs : $N_{\text{packs}} = \left\lceil \frac{N}{64} \right\rceil$ cofres dobles : $N_{\text{dc}} = \left\lceil \frac{N}{3456} \right\rceil$

Un *cofre simple* guarda $27 \times 64 = 1\,728$ ítems. Una *shulker box* (contenedor portátil) guarda $27 \times 64 = 1\,728$ ítems pero puede a su vez almacenarse *dentro* de un slot de cofre, así que un cofre doble lleno de 54 shulker boxes guarda hasta $54 \times 1\,728 = 93\,312$ ítems. Para mega-builds la unidad relevante es el *cofre doble cargado de shulkers*.

17.3 Tabla de referencia: esferas canónicas

La tabla convierte los volúmenes canónicos de esfera rellena en planes de recolección. La última columna sugiere un contexto de construcción para cada diámetro:

Diámetro	V (bloques)	Packs	Almacenamiento	Construcción típica
D=8	268	5	1 cd	techo de cabaña, cúpula de pozo
D=12	904	15	1 cd	cima de atalaya
D=16	2145	34	1 cd	campana de aldea, cúpula de biblioteca
D=20	4189	66	2 cd	cúpula de observatorio
D=24	7238	114	3 cd	cúpula de templo
D=32	17156	269	5 cd	cúpula de catedral, arena de world-boss

La representación *stacks-y-resto* $N = q \cdot 64 + r$ que muestra el chip de info refleja cómo el inventario de supervivencia exhibe stacks parciales: q slots llenos más un slot mostrando r . Para $D = 16$, el chip imprime «2 145 ($33 \times 64 + 33$)» — 33 slots completos más uno con 33 ítems, exactamente 34 slots, una fila más cuatro slots.

18. Highlight overlay y conteo de caras expuestas

Block Round renderiza por defecto un *contorno negro* (highlight overlay) en las aristas de cada cara de voxel visible. Visualmente recuerda al estilo «F3+G» de fronteras de chunk del overlay de depuración, escalado al nivel por-voxel. Matemáticamente implementa la visualización del **conteo de caras expuestas**: un número que los constructores de supervivencia usan para validar progreso bloque a bloque.

18.1 Qué cuenta como cara expuesta

Una cara del voxel $\mathbf{v}$ compartida con el vecino $\mathbf{n}$ está *expuesta* si y solo si $\mathbf{n} \notin V_{\text{sólido}}$ (el vecino está vacío). Cada voxel sólido aporta entre 0 (totalmente enterrado) y 6 (aislado) caras. El total F_{exp} es el análogo discreto del *área de superficie*.

18.2 Fórmula de detección de borde

Un voxel está en el borde si al menos uno de sus seis vecinos-cara está fuera del sólido:

$$b(i, j, k) = f(i, j, k) \wedge (\neg f(i \pm 1, j, k) \vee \neg f(i, j \pm 1, k) \vee \neg f(i, j, k \pm 1))$$

Es el mismo operador de borde del algoritmo fino 2D extendido a 3D, y el overlay simplemente dibuja las líneas de arista cúbica de cada voxel-borde. Para Vidrio y Hielo el valor por defecto es OFF; para Adoquín, Piedra, Diamante y otros bloques opacos es ON, porque el overlay desambigua voxels internos de los de la cáscara.

18.3 Conteo de caras expuestas como heurística de supervivencia

Para constructores de supervivencia la cuenta relevante no es el volumen V sino las *caras expuestas* F_{exp} , porque eso es lo visible desde fuera. La cuenta total satisface seis veces los voxels-borde menos el doble de las adyacencias-cara entre voxels-borde:

$$F_{\text{exp}} = \sum_{\mathbf{v} \in V_{\text{sólido}}} \sum_{\mathbf{n} \in N_6(\mathbf{v})} [\mathbf{v} + \mathbf{n} \notin V_{\text{sólido}}]$$

Para una esfera $D = 16$ (modo relleno), $V = 2145$, $V_{\text{borde}} \approx 432$, $F_{\text{exp}} \approx 1184$ caras visibles — la construcción necesita solo unos 432 bloques *visibles*; los 1713 restantes son interior y pueden rellenarse con el bloque más barato (Tierra, Adoquín). El overlay permite detectar al instante un bloque ausente en la cáscara.

19. Construcción capa por capa (descomposición horizontal)

En Minecraft la construcción avanza *por capas horizontales*: el jugador está en una capa, coloca los bloques de la capa superior, luego sube un bloque (salta + coloca andamiaje temporal o usa élitros). El problema 3D del elipsoide se entiende mejor como una *pila de elipses 2D*, una por capa:

$$\text{capa}(k) = \{ (i, j) : a_x(i)^2 + a_y(j)^2 \leq 1 - a_z(k)^2 \}$$

Para cada capa entera k de $-r_z$ a $+r_z$, la sección transversal es una elipse con semi-ejes encogidos $r_x(k)$ y $r_y(k)$:

$$r_x(k) = r_x \sqrt{1 - \left(\frac{k}{r_z}\right)^2}, \quad r_y(k) = r_y \sqrt{1 - \left(\frac{k}{r_z}\right)^2}$$

Esto reduce el problema 3D a *computar el algoritmo 2D de las secciones anteriores, una capa a la vez*, y apilar resultados. Cognitivamente es una simplificación masiva: en vez de tres coordenadas mentales el constructor maneja dos por capa más el índice de capa.

Mapeo a Minecraft. Para una esfera $D = 16$ ($r_z = 8$), la capa ecuatorial ($k = 0$) es el círculo Bresenham $D = 16$ (≈ 200 bloques). La siguiente ($k = 1$) es un círculo $D = 15,97$, casi la misma forma sin las celdas de esquina. En $k = 4$ la capa es un círculo $D = 13,86$ (≈ 148 bloques); en $k = 7$ ya es $D = 7,48$ (≈ 43 bloques). Los polos ($k = \pm 8$) son una tapa de un bloque.

El chip de info de Block Round expone esta cuenta por capa en modo 3D cuando el usuario activa el botón de Guías centrales (tecla **c**). Las rebanadas horizontales se dibujan como una cruz amarilla translúcida en el ecuador y en cada subdivisión $r_z/2$.

20. Optimización por simetría octaédrica (O_h)

Una esfera centrada en el origen es invariante bajo la acción del **grupo octaédrico** O_h de orden 48. La voxelización preserva un subgrupo de orden 48 generado por las tres reflexiones coordenadas y las tres rotaciones de $\pi/2$ en torno a cada eje. Para fines prácticos de construcción, el subgrupo relevante es el grupo de reflexiones $\mathbb{Z}_2^3$ de orden 8, generado por $x \mapsto -x$, $y \mapsto -y$, $z \mapsto -z$. El conjunto de voxels en un octante determina la esfera completa:

$$V_{\text{total}} \approx 8 V_{\text{octant}} \implies \text{reducción} = \frac{N - N/8}{N} = 87,5\%$$

Es la mayor optimización matemática disponible para un constructor de Minecraft. En vez de colocar manualmente N bloques, el constructor coloca $N/8$ en el octante positivo y emite siete comandos `/clone` (vanilla) o `//copy + //paste` (WorldEdit) con las reflexiones apropiadas.

20.1 Secuencia concreta de comandos

Para una esfera $D = 24$ ($r = 12$, $V = 7\,238$ bloques) centrada en $(0, 72, 0)$, construye el octante positivo primero ($+x, +y, +z$, ≈ 905 bloques), después emite:

```
/clone 0 72 0 12 84 12 ~~~ filtered minecraft:stone
/clone 0 72 0 12 84 12 -12 72 0 mode:masked (reflex. -x)
/clone 0 72 0 12 84 12 0 60 0 mode:masked (reflex. -y)
/clone 0 72 0 12 84 12 0 72 -12 mode:masked (reflex. -z)
/clone -12 72 0 -1 84 12 mode:masked (octante -x, -y)
/clone 0 60 -12 12 71 -1 mode:masked (octante -y, -z)
/clone -12 72 -12 -1 84 -1 mode:masked (octante -x, -z)
```

```
/clone -12 60 -12 -1 71 -1 mode:masked (octante  $-x, -y, -z$ )
```

Cada `/clone` cuesta unos 50 ms de tiempo de servidor, así las siete reflexiones completan en menos de medio segundo. La colocación manual de los 905 bloques del octante lleva unos 15 min en supervivencia o 45 s en creativo con brush. La colocación completa de 7 238 bloques sin simetría llevaría ≈ 2 h en supervivencia o 6 min en creativo.

20.2 Comparación de tiempo

Sea T_{block} el tiempo de colocación por bloque (típicamente 1–2 s en supervivencia, 0,1–0,5 s en creativo) y T_{clone} el tiempo por comando (≈ 50 ms). El tiempo total con y sin simetría es:

$$T_{\text{octant}} + 7 T_{\text{clone}} \ll T_{\text{full}}$$

Para $N = 7\,238$ en supervivencia ($T_{\text{block}} = 2$ s, $T_{\text{clone}} = 0,05$ s): completo = 14 476 s ≈ 4 h 1 min; octante + clones = 1 810 s + 0,35 s ≈ 30 min. Un factor-8 que casa con el orden del subgrupo de reflexiones. El O_h completo de orden 48 podría reducir aún más, pero exige una zona rotada 45° , raramente vale el coste de setup en Minecraft vanilla.

21. Texturas de bloques y composición visual

Un bloque de Minecraft no es una celda plana coloreada sino un *cubo con seis caras texturizadas*. Block Round renderiza cada voxel con sus seis texturas canónicas, muestreadas del resource pack vanilla. Matemáticamente la texturización es un *mapeo UV*: cada cara del cubo unitario se parametriza por coordenadas $[0, 1]^2$, y la textura se muestrea en esas UVs para producir los píxeles renderizados.

21.1 Las seis caras y el mapeo UV

Para cada cara de voxel el renderer consulta el modelo del bloque:

$$\text{cara} \in \{\text{techo, lado, fondo}\} \quad \text{UV} : [0, 1]^2 \rightarrow \text{pixels}$$

La mayoría de bloques (Piedra, Adoquín, Bloque de Diamante, Tablones de Roble) usan la *misma textura en todas las caras*. Los bloques multi-cara (Bloque de Hierba, Calabaza, Heno, Pilar de Cuarzo, todos los troncos, Mesa de Trabajo, Horno, Estantería, TNT) usan *texturas distintas* en techo, lados y fondo: Hierba tiene techo verde, lado hierba-y-tierra y fondo de tierra; Tronco de Roble tiene

el anillo de madera en techo/fondo y corteza en los cuatro lados. Lo importante: la *matemática de qué voxels colocar* no depende de la textura — la silueta de una esfera de Diamante y una de Adoquín del mismo diámetro es idéntica. La decisión de textura es una capa *puramente estética* aplicada después del algoritmo geométrico.

21.2 Familias de bloques y tonos

Para el constructor de supervivencia eligiendo entre las decenas de bloques disponibles, la pregunta relevante no es el índice de textura sino la *familia tonal*: claro/oscuro, cálido/frío, uniforme/estampado. La tabla clasifica los bloques más usados en construcciones Block Round:

Bloque	Familia	Tono	Uso típico
cobblestone	piedra	gris	muros rústicos, mazmorras
stone	piedra	gris	muros limpios, pedestales
oak_planks	madera	beige	pisos, acabado interior
quartz_block	cuarzo	blanco	templos, cúpulas modernas
sandstone	arena	beige	desiertos, pirámides
deepslate	piedra	oscuro	subterráneos, acentos

21.3 Gradientes por mezcla de bloques

Aunque un tipo único de bloque proporciona un tono, los *gradientes* se obtienen intercalando dos o tres tipos por capa. Una técnica clásica es el gradiente *piedra-a-cuarzo* en una cúpula: capas inferiores Piedra Lisa, capas ecuatoriales una mezcla ruidosa de Piedra Lisa y Diorita, capas superiores Cuarzo. Cada capa sigue calculándose por el mismo algoritmo Block Round; lo que cambia es la consulta de textura por voxel. La opción *Random* de Block Round implementa una mezcla procedural así (techo de hierba, tierra debajo, minerales esparcidos, deepslate cerca de la bedrock). La matemática es la misma de la descomposición por capa de la Sección 19; solo cambia la «etiqueta de bloque» por voxel.

22. Conclusión

Este documento describió, de forma sistemática, los fundamentos matemáticos empleados en Block Round. En aproximadamente mil líneas de JavaScript más los pipelines de textura y audio, el proyecto integra contribuciones de varias áreas teóricas:

- **Geometría analítica:** ecuaciones implícitas de la elipse y el elipsoide como criterio primario de pertenencia.
- **Algoritmos clásicos de rasterización:** Bresenham en formulación de punto medio, con extensión de Pitteway/Kappel para elipses.
- **Topología digital:** operadores de borde discreto, conceptos de 4-, 6- y 26-conectividad, composición por rebanadas y conteo de caras expuestas.
- **Cálculo integral:** volumen del elipsoide como integral triple y su contraparte discreta (medida de conteo sobre voxels).
- **Álgebra lineal:** planos de corte definidos por desigualdades lineales y proyección perspectiva tridimensional.
- **Trigonometría:** parametrización de la cámara en coordenadas esféricas y ajuste automático de distancia en función del FOV.
- **Aritmética combinatoria de inventario:** contabilidad por división con techo de packs de 64 y cofres dobles de 3456 ítems, con la representación stacks-y-resto del chip de info.
- **Simetría de grupos:** explotación del grupo octaédrico O_h y su subgrupo de reflexiones de orden 8 $\mathbb{Z}_2^3$ para reducir el coste de construcción por un factor de 8 vía `/clone`.

La observación central que el estudio permite formular es: en una malla discreta, **no existe una representación única correcta** de una curva continua. Cada algoritmo aquí presentado corresponde a una definición matemática distinta del criterio de inclusión de celdas, y cada definición produce una silueta con propiedades formales propias — suavidad en Euclidiano, patrón escalonado de Bresenham, mayor cobertura en cobertura-por-esquina. La elección del algoritmo es una decisión técnica que debe considerar la representación visual deseada, las características métricas buscadas y — específicamente para Block Round — el coste de inventario y de tiempo de construir físicamente la forma in-game.

Block Round ocupa un nicho pequeño pero matemáticamente rico en el ecosistema de herramientas Minecraft: se sitúa entre las puramente geométricas (WorldEdit, Litematica) y las puramente visuales (resource packs, shaders), ofreciendo un puente explícito, derivable y reproducible de la ecuación continua al plan de colocación discreto. El proyecto hermano Pixel Round ofrece los mismos algoritmos sin la capa Minecraft, indicado para pixel art y voxel art tradicional. Juntos los dos proyectos ejemplifican cómo el mismo canal matemático continuo-a-discreto sostiene flujos artísticos y constructivos radicalmente distintos.

Apéndice A: Tablas de referencia de construcción y ejemplo práctico

Este apéndice reúne las referencias numéricas y de comandos más usadas por usuarios de Block Round. Los datos consolidan las tablas por sección y los ejemplos prácticos repartidos por el documento, organizados para que el apéndice funcione como consulta de una página para planificación, sintaxis de comandos y un ejemplo práctico de extremo a extremo.

A.1 Referencia completa de esferas (rellena, hueca, packs)

La tabla extiende las tablas canónicas de esfera de las Secciones 10 y 17 añadiendo la variante hueca (modo *fino*) junto al volumen relleno. Las cuentas de cáscara hueca asumen un solo voxel de grosor y se computan bajo el algoritmo Euclidiano; el modo grueso añade aproximadamente 10–15% más bloques para sellar las grietas diagonales según la Sección 12.

D	Rellena (V)	Hueca (cáscara)	Packs rellena	Packs hueca
8	268	170	5	3
10	523	316	9	5
12	904	500	15	8
14	1437	744	23	12
16	2145	1042	34	17
18	3052	1338	48	21
20	4189	1648	66	26
24	7238	2400	114	38
28	11494	3296	180	52
32	17156	4332	269	68

Para un constructor planificando una partida de supervivencia, la columna *hueca* suele ser la relevante: solo la cáscara visible necesita el bloque texturizado dedicado, mientras el interior (si se desea alguno) puede rellenarse con el material más barato. La columna *packs-rellena* indica el tamaño total de reservas para una esfera completamente rellena; *packs-hueca* es el mínimo realista para una cúpula o cáscara hueca.

A.2 Referencia de comandos WorldEdit / vanilla

La tabla resume los comandos WorldEdit y de Minecraft vanilla más relevantes para construir las salidas de Block Round in-game. Los comandos WorldEdit

asumen que el jugador lleva una varita (hacha de madera por defecto) y ha seleccionado una región; los vanilla funcionan sin mods pero requieren argumentos de coordenadas.

Comando	Mod / origen	Resultado
<code>//sphere stone 8</code>	WorldEdit	esfera rellena $r=8$ (Euclidiana)
<code>//hsphere stone 8</code>	WorldEdit	esfera hueca $r=8$ (solo cáscara)
<code>//cyl stone 8 16</code>	WorldEdit	cilindro $r=8$, $h=16$
<code>//ellipsoid st. 10 5 10</code>	WorldEdit	elipsoide $10 \times 5 \times 10$
<code>/fill ... stone</code>	vanilla	rellenar caja con un tipo de bloque
<code>/clone ... mode:masked</code>	vanilla	copiar región a nueva ubicación

Para usuarios de Litematica el flujo es distinto: en lugar de ejecutar comandos, el archivo schematic (Block Round exporta `.schem`, formato Sponge v2) se carga en el mod Litematica, que muestra un fantasma translúcido de la figura que el jugador coloca bloque a bloque. Es el equivalente supervivencia más cercano a la experiencia creativa de WorldEdit.

A.3 Ejemplo práctico: cúpula $D=20$ de Cuarzo sobre un templo

Como ejemplo práctico completo, suponga que un constructor quiere una cúpula de Cuarzo de $D = 20$ sobre un templo de piedra existente. La parte superior del templo es 11×11 bloques, centrada en $(0, 80, 0)$. La cúpula es la mitad superior de una esfera de radio $r = 10$ cortada en el plano $Y = 0$. El plan es:

1. **Planear con Block Round.** Abrir la app, modo 3D, Esfera, tamaño $D = 20$, eje de corte Y al 50%. Seleccionar Bloque de Cuarzo. El chip de info reporta $V_{\text{cúpula}} \approx 2\,126$ bloques (≈ 34 packs, 1 cofre doble con 1 330 ítems sobrantes en el segundo).
2. **Reunir recursos.** 34 packs de Bloque de Cuarzo $= \lceil 2\,126/4 \rceil = 532$ Bloques de Cuarzo $\times 4$ Cuarzo del Nether cada uno $= 2\,128$ Cuarzos del Nether. Fundir o comerciar según convenga.
3. **Elegir algoritmo.** Para una cúpula vista desde abajo, el algoritmo Euclidiano da la silueta más suave; para una cúpula en una montaña remota el algoritmo Umbral se lee más limpio. Block Round usa Euclidiano por defecto.
4. **Posicionar la cúpula.** La cara plana debe coincidir con la parte superior del templo. El bounding box es $20 \times 10 \times 20$ bloques; colocar su centro en $(0, 80, 0)$, de modo que la cara plana esté en $y = 80$ y el ápice en $y = 89$.

5. **Construir por simetría.** Construir el cuadrante $+x, +z$ (≈ 530 bloques). Ejecutar `/clone 0 80 0 10 89 10 -10 80 0 mode:masked` para reflejar en x ; luego `/clone -10 80 0 10 89 10 0 80 -10 mode:masked` para reflejar en z . Tiempo total: ~ 10 minutos en supervivencia.
6. **Pulir.** Añadir Linternas Marinas en el ápice y los cuatro puntos cardinales del ecuador para iluminación emisiva (Block Round las renderiza con su mapa emisivo al nivel de luz MC 15). Opcionalmente alternar el overlay de borde (tecla **G**) para verificar la cáscara antes de colocar los últimos bloques.

Las matemáticas de cada paso son exactamente lo que las veintidós secciones anteriores describen: ecuación implícita del elipsoide, voxelización Euclidiana, corte en Y , reducción por simetría octaédrica vía `/clone`, aritmética de inventario para traducir el volumen en packs y cofres. El apéndice no es por tanto teoría nueva — es un recordatorio de que cada concepto teórico del documento mapea a una decisión concreta única en el flujo de planificación de construcción.

A.4 Referencia de atajos de teclado

La app Block Round expone un pequeño conjunto de atajos de teclado que reflejan los botones de esquina del canvas. Se listan abajo para referencia rápida; los atajos son globales (no se necesita ningún campo de input enfocado) y funcionan en modos 2D y 3D salvo indicación contraria.

Tecla	Alternativa	Notas
G	Overlay de borde	contorno negro en cada cara de voxel
C	Guías centrales	cruz amarilla translúcida en los ejes
D	Descargar PNG	canvas actual como imagen PNG
I	Chip de info	conteo de bloques y desglose de packs
M	Alternar 2D / 3D	cambia entre modo plano y voxel
S	Sonido on/off	sonidos ambientales de colocación
T	Tema día / noche	oscurece el fondo, mantiene piezas legibles
Ctrl+Z / Y	Deshacer / rehacer	recorre el historial de cambios de la figura

Los atajos **G** (cuadrícula) y **C** (guías centrales) son especialmente útiles durante la validación de la construcción: **G** muestra si cada voxel de la cáscara está en su lugar (un bloque ausente rompe el patrón del overlay), y **C** confirma que los cortes están bien centrados en los ejes de la figura.

Para el **Ctrl+Z** de deshacer: Block Round recorre toda edición que cambia la figura (sliders, modo de render, algoritmo, forma, eje, bloque, alternancia de modo) pero

excluye deliberadamente alternancias solo visuales (cámara, overlay de bordes, cruz central, tema, sonido). Esta separación mantiene la pila de undo poblada con ediciones que el usuario probablemente querrá revertir.

A.5 Glosario de términos clave

Referencia rápida de los términos técnicos que se repiten por este documento. Cada entrada recapitula brevemente el concepto y apunta a la sección donde se desarrolla en detalle.

- **Voxel** — cubo unitario en 3D, análogo del píxel en 2D. Cada bloque de Minecraft es un voxel. Ver Sección 2.
- **Pack (stack)** — en Minecraft, una pila de 64 ítems idénticos que ocupa un slot del inventario. Ver Sección 17.
- **Cofre doble** — dos cofres adyacentes fusionados en un contenedor de 54 slots. Almacena hasta 3 456 ítems. Ver Sección 17.
- **Shulker box** — contenedor portátil de 27 slots que puede a su vez almacenarse dentro de un slot de cofre. Ver Sección 17.
- **Cáscara** — la capa exterior de un voxel de grosor de una figura sólida. La superficie relevante para constructores de supervivencia. Ver Secciones 7 y 18.
- **Octante** — una de las ocho regiones del espacio 3D obtenidas por la intersección de los tres semiespacios coordenados. Usada en optimización por simetría. Ver Sección 20.
- **Mapeo UV** — una parametrización 2D de una cara 3D, usada para mapear una imagen de textura en cada cara de un voxel. Ver Sección 21.
- **Bresenham** — algoritmo incremental de rasterización de 1965 que usa solo aritmética entera. Ver Sección 5.
- **Umbral** — criterio de rasterización que incluye una celda si alguno de sus ángulos está dentro de la elipse. Ver Sección 6.
- **Euclidiano** — criterio de rasterización que incluye una celda si su centro está dentro de la elipse. Ver Sección 4.
- **Easter egg** — función oculta activada por una entrada específica. En Block Round: el roble y el creeper, ambos en el valor 15 del slider.
- **Schematic** — archivo de estructura serializada (Sponge Schematic v2, `.schem`)

cargable por WorldEdit, Litematica o MCEdit.

Este glosario es deliberadamente más corto que un léxico matemático completo. Lectores buscando cobertura más profunda de la matemática subyacente (topología digital, cálculo integral, teoría de grupos, etc.) son remitidos a las secciones relevantes de este documento.

A.6 Perspectivas y límites del modelo

El modelo Block Round presentado en este documento cubre la voxelización estática de elipses, elipsoides y sus cortes, además de las consideraciones prácticas introducidas por la capa Minecraft. No aborda problemas dinámicos — figuras en movimiento, transiciones de voxel animadas, interacciones con simulación de fluidos — porque requieren un marco de tiempo continuo fuera del alcance de la rasterización discreta. Una extensión natural es la *animación a nivel de schematic*: una secuencia de voxelizaciones estáticas conectadas por interpolación, que Block Round ya puede exportar como un flujo `.schem` legible por scripts de Litematica.

Otra dirección abierta es la *voxelización exacta que preserva volumen*: el conteo discreto actual V_{disc} converge al volumen continuo V_{cont} solo asintóticamente. Para aplicaciones donde el conteo exacto debe coincidir con la fórmula continua (p.ej. competiciones de render, arte matemático con presupuestos estrictos de bloques), se requiere una capa extra de refinamiento — por ejemplo, ajustando ligeramente el radio para que el conteo discreto alcance el valor objetivo. Block Round no implementa actualmente este refinamiento, pero la matemática es directa: resolver $V_{\text{disc}}(r) = V_{\text{objetivo}}$ numéricamente para r .

Finalmente, la *explotación de simetría de orden superior*: el presente documento cubre el subgrupo de reflexiones de orden 8 de O_h , pero el grupo completo de orden 48 incluye rotaciones que en principio permitirían un aceleramiento de $\times 48$. El obstáculo práctico es que los comandos in-game operan en regiones alineadas a los ejes, y explotar las simetrías rotacionales requiere un área de construcción rotada 45° . Mods de servidor que rotan internamente las cajas de selección (p.ej. FAWE) pueden en principio lograrlo, pero el tooling mainstream aún no lo expone. Una implementación de referencia demostrando $\times 48$ en un servidor vanilla sería un seguimiento natural al presente trabajo.

A.7 Referencias adicionales y recursos externos

Los conceptos matemáticos desarrollados en este documento tienen literaturas bien establecidas. Para lectores que buscan fuentes primarias o mayor profundidad algorítmica, se recomiendan los siguientes puntos de partida:

- J. E. Bresenham, *Algorithm for computer control of a digital plotter*, IBM Systems Journal, vol. 4 no. 1, 1965 — el artículo original de rasterización por aritmética entera extendido en este documento.
- M. L. V. Pitteway, *Algorithm for drawing ellipses or hyperbolae with a digital plotter*, Computer Journal, vol. 10, 1967 — generaliza Bresenham a cónicas arbitrarias.
- A. Kappel, *An ellipse-drawing algorithm for raster displays*, ACM Comm. vol. 28, 1985 — la formulación elíptica de dos regiones usada en la Sección 5.
- R. Klette y A. Rosenfeld, *Digital Geometry: Geometric Methods for Digital Picture Analysis*, Morgan Kaufmann, 2004 — referencia estándar para topología digital y los operadores de borde de las Secciones 7 y 18.
- Documentación de WorldEdit, enginehub.org/worldedit/ — referencia oficial de los comandos `//sphere`, `//hsphere`, `//ellipsoid`, `//copy` usados a lo largo de este documento.
- Especificación Sponge Schematic v2, github.com/SpongePowered/Schematic-Specification — el formato de archivo que Block Round exporta como `.schem`.

Las ocho áreas teóricas listadas en la Sección 22 tienen tratamientos a nivel de libro de texto que van mucho más allá de lo que un solo documento técnico puede resumir. La contribución de Block Round no está en la teoría misma sino en la síntesis específica: aplicar rasterización clásica al escenario de Minecraft de voxels texturizados, con las consideraciones explícitas de inventario y coste de simetría que usualmente se omiten en herramientas puramente geométricas.

Un comentario final sobre reproducibilidad: todo el pipeline de este documento — script de build, traducciones, plantilla LaTeX, tablas por locale — está contenido en el directorio `docs_math/` del repositorio del proyecto. Re-ejecutar `python build.py` en un sistema con XeLaTeX/MiKTeX produce este PDF exactamente, en cualquiera de los nueve locales soportados. El proyecto hermano Pixel Round sigue la misma estructura, permitiendo comparación directa lado a lado de las variantes texturizada y no texturizada.

Fin del documento

Block Round es mantenido por Vinícius Rodrigues de Souza. El proyecto hermano Pixel Round está disponible en github.com/ViniSouza128/pixel-round; el repositorio actual de Block Round está en github.com/ViniSouza128/block-round.

Comentarios, correcciones y mejoras de traducción son bienvenidos vía el issue tracker del repositorio del proyecto. La revisión por hablantes nativos de las ocho locales no inglesas es particularmente valiosa, ya que las localizaciones se prepararon con revisión sustancial pero la terminología técnico-matemática se beneficia de corrección nativa.